你說：

我要寫一個java骰子模擬器你可以根據這個程式碼去延伸功能嗎 例如連骰十次之類的 然後我點數六要是綠色 點數一紅色

ChatGPT 說：

可以！我們可以把原本的 Java 骰子程式延伸成一個完整的 **Java Swing 骰子模擬器**，不只是按一下出現一個數字，還可以加入統計功能。

我記得你之前希望骰子是有實際骰子外觀的圖形，而不是單純顯示數字，所以可以沿用 `JFrame`、`JPanel`、`JButton`，搭配 `Graphics` 自己繪製骰子。

一、預計增加的功能

骰子模擬器

6	1	2–5
綠色	紅色	黑色

操作按鈕

擲一次 連骰十次 重新開始

統計資訊

- 擲骰總次數
- 點數總和
- 平均點數
- 1～6 各點數出現次數

二、我建議的程式功能

功能	說明
單次擲骰	每按一次產生 1～6
連骰十次	一次執行十次擲骰

顏色判斷	6 綠色、1 紅色、其他黑色
次數統計	計算總共擲了幾次
點數統計	計算每個點數出現幾次
平均點數	點數總和 ÷ 擲骰次數
重新開始	清空統計紀錄

三、先把你的原始程式碼給我

請把你目前的 Java 骰子程式碼貼上來。

我會直接以你的原始程式為基礎修改，而不是重新寫一套完全不同的程式。

另外會盡量保留你原本學過的寫法，使用：

- `JFrame` ：建立視窗
- `JPanel` ：放置骰子及元件
- `JButton` ：操作按鈕
- `addActionListener` ：按鈕事件
- `Random` ：產生隨機點數
- `Graphics` ：繪製骰子

我也會幫你把完整程式碼整理好，並逐段解釋新增功能的用途，讓你之後能自己修改，而不只是複製貼上。

你說：

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class HW2 extends JFrame {

    private JLabel infoLabel;
    private DicePanel dicePanel;

    private int count = 0;
    private int sum = 0;

    private Random random = new Random();
```

```
public HW2() {

    // ===== 視窗設定 =====
    setTitle("骰子模擬器");
    setSize(400, 320);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null); // 視窗置中

    setLayout(new BorderLayout());

    // ===== 上方資訊 =====
    infoLabel = new JLabel(
            "已擲 0 次，總和 0，平均 0.00",
            JLabel.CENTER
    );

    infoLabel.setFont(
            new Font("Microsoft JhengHei", Font.PLAIN, 16)
    );

    add(infoLabel, BorderLayout.NORTH);


    // ===== 中間骰子 =====
    dicePanel = new DicePanel();

    add(dicePanel, BorderLayout.CENTER);


    // ===== 下方按鈕 =====
    JButton rollButton = new JButton("擲骰子");

    rollButton.setFont(
            new Font("Microsoft JhengHei", Font.BOLD, 18)
    );

    rollButton.setPreferredSize(new Dimension(120, 45));

    // 按下按鈕
    rollButton.addActionListener(e -> rollDice());

    JPanel buttonPanel = new JPanel();
```

```
        buttonPanel.add(rollButton);

        add(buttonPanel, BorderLayout.SOUTH);

        setVisible(true);
    }


    // ===== 擲骰子 =====
    private void rollDice() {

        // 產生 1～6
        int dice = random.nextInt(6) + 1;

        // 更新資料
        count++;
        sum += dice;

        // 更新骰子
        dicePanel.setNumber(dice);

        // 計算平均
        double average = (double) sum / count;

        // 更新上方文字
        infoLabel.setText(
                String.format(
                        "已擲 %d 次，總和 %d，平均 %.2f",
                        count,
                        sum,
                        average
                )
        );
    }


    //
================================================
===
    // 自訂骰子面板
    //
================================================
===
```

```
class DicePanel extends JPanel {

    private int number = 1;

    public DicePanel() {
        setBackground(Color.WHITE);
    }

    public void setNumber(int number) {
        this.number = number;
        repaint();
    }


    @Override
    protected void paintComponent(Graphics g) {

        super.paintComponent(g);

        Graphics2D g2 = (Graphics2D) g.create();

        // 開啟反鋸齒
        g2.setRenderingHint(
                RenderingHints.KEY_ANTIALIASING,
                RenderingHints.VALUE_ANTIALIAS_ON
        );

        // ===== 骰子大小 =====
        int size = 130;

        int x = (getWidth() - size) / 2;
        int y = (getHeight() - size) / 2;

        // ===== 骰子陰影 =====
        g2.setColor(new Color(180, 180, 180));

        g2.fillRoundRect(
                x + 6,
                y + 8,
                size,
                size,
                30,
```

```
        30
);


// ===== 骰子本體 =====
g2.setColor(Color.WHITE);

g2.fillRoundRect(
        x,
        y,
        size,
        size,
        30,
        30
);


// ===== 骰子邊框 =====
g2.setColor(new Color(150, 150, 150));

g2.setStroke(new BasicStroke(2));

g2.drawRoundRect(
        x,
        y,
        size,
        size,
        30,
        30
);


// ===== 點的設定 =====
int pipSize = 20;

int left = x + 35;
int center = x + size / 2 - pipSize / 2;
int right = x + size - 55;

int top = y + 30;
int middle = y + size / 2 - pipSize / 2;
int bottom = y + size - 50;
```

```
// 黑色骰子點
g2.setColor(Color.BLACK);


// ===== 根據點數畫點 =====

switch (number) {

    case 1:
        drawPip(g2, center, middle, pipSize);
        break;

    case 2:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, right, bottom, pipSize);
        break;

    case 3:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, center, middle, pipSize);
        drawPip(g2, right, bottom, pipSize);
        break;

    case 4:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, right, top, pipSize);
        drawPip(g2, left, bottom, pipSize);
        drawPip(g2, right, bottom, pipSize);
        break;

    case 5:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, right, top, pipSize);
        drawPip(g2, center, middle, pipSize);
        drawPip(g2, left, bottom, pipSize);
        drawPip(g2, right, bottom, pipSize);
        break;

    case 6:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, right, top, pipSize);
```

```
                drawPip(g2, left, middle, pipSize);
                drawPip(g2, right, middle, pipSize);

                drawPip(g2, left, bottom, pipSize);
                drawPip(g2, right, bottom, pipSize);
                break;
            }

            g2.dispose();
        }


        // ===== 畫骰子上的一個點 =====
        private void drawPip(
                Graphics2D g2,
                int x,
                int y,
                int size
        ) {

            g2.fillOval(x, y, size, size);
        }
    }


    // ===== 主程式 =====
    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
            new HW2();
        });
    }
}
```

ChatGPT 說：

我看過你的原始程式碼了！你的程式架構其實已經很完整，不需要整個重寫。

我會保留你原本的 `HW2`、`DicePanel`、`rollDice()` 和骰子繪圖方式，再往上增加功能。

這次我幫你規劃成一個比較完整的版本：

一、這次增加的功能

1. **擲骰子**：每次按下按鈕，隨機產生 1～6。
2. **連骰十次**：一次自動擲骰十次，顯示最後一次的骰子結果。
3. **顏色變化**：
   - 點數 1：紅色。
   - 點數 6：綠色。
   - 點數 2～5：黑色。
4. **次數統計**：紀錄總共擲了幾次。
5. **點數統計**：分別紀錄 1～6 出現幾次。
6. **平均點數**：計算所有擲骰結果的平均。
7. **重新開始**：將所有統計數據歸零。

另外，我會讓連骰十次真正執行十次隨機產生，而不是只產生一次結果再把次數加十。

二、完整 Java 程式碼

以下可以直接取代你原本的程式碼，檔名維持 `HW2.java`。

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class HW2 extends JFrame {

    private JLabel infoLabel;
    private JLabel statsLabel;
    private DicePanel dicePanel;

    private int count = 0;
    private int sum = 0;

    // 紀錄 1～6 各出現幾次
    private int[] frequency = new int[7];

    private Random random = new Random();

    public HW2() {
```

```
        // ===== 視窗設定 =====
        setTitle("骰子模擬器");
        setSize(500, 480);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        setLayout(new BorderLayout(10, 10));


        // ===== 上方資訊 =====
        infoLabel = new JLabel(
                "已擲 0 次，總和 0，平均 0.00",
                JLabel.CENTER
        );

        infoLabel.setFont(
                new Font("Microsoft JhengHei", Font.BOLD, 18)
        );

        add(infoLabel, BorderLayout.NORTH);


        // ===== 中間骰子 =====
        dicePanel = new DicePanel();

        add(dicePanel, BorderLayout.CENTER);


        // ===== 下方按鈕 =====

        JButton rollButton = new JButton("擲骰子");
        JButton tenButton = new JButton("連骰十次");
        JButton resetButton = new JButton("重新開始");

        rollButton.addActionListener(e -> rollDice());

        tenButton.addActionListener(e -> rollTenTimes());

        resetButton.addActionListener(e -> resetGame());
```

```
// 按鈕字體
Font buttonFont = new Font(
        "Microsoft JhengHei",
        Font.BOLD,
        15
);

rollButton.setFont(buttonFont);
tenButton.setFont(buttonFont);
resetButton.setFont(buttonFont);


// 按鈕面板
JPanel buttonPanel = new JPanel();

buttonPanel.add(rollButton);
buttonPanel.add(tenButton);
buttonPanel.add(resetButton);


// ===== 點數統計 =====
statsLabel = new JLabel(
        "1點：0次  2點：0次  3點：0次"
        + "  4點：0次  5點：0次  6點：0次",
        JLabel.CENTER
);

statsLabel.setFont(
        new Font("Microsoft JhengHei", Font.PLAIN, 13)
);


// 將按鈕與統計放在下方
JPanel bottomPanel = new JPanel(new GridLayout(2, 1));

bottomPanel.add(buttonPanel);
bottomPanel.add(statsLabel);

add(bottomPanel, BorderLayout.SOUTH);
```

```
        setVisible(true);
    }


    // ===== 單次擲骰子 =====
    private void rollDice() {

        int dice = random.nextInt(6) + 1;

        updateData(dice);
    }


    // ===== 連續擲骰十次 =====
    private void rollTenTimes() {

        for (int i = 0; i < 10; i++) {

            int dice = random.nextInt(6) + 1;

            updateData(dice);
        }
    }


    // ===== 更新統計資料 =====
    private void updateData(int dice) {

        // 更新總次數
        count++;

        // 更新點數總和
        sum += dice;

        // 更新各點數出現次數
        frequency[dice]++;

        // 更新骰子畫面
        dicePanel.setNumber(dice);

        // 計算平均
```

```
        double average = (double) sum / count;

        // 更新上方資訊
        infoLabel.setText(
                String.format(
                        "已擲 %d 次，總和 %d，平均 %.2f",
                        count,
                        sum,
                        average
                )
        );

        // 更新各點數統計
        statsLabel.setText(
                "1點：" + frequency[1] + "次  "
                + "2點：" + frequency[2] + "次  "
                + "3點：" + frequency[3] + "次  "
                + "4點：" + frequency[4] + "次  "
                + "5點：" + frequency[5] + "次  "
                + "6點：" + frequency[6] + "次"
        );
    }


    // ===== 重新開始 =====
    private void resetGame() {

        count = 0;
        sum = 0;

        // 將所有點數統計歸零
        for (int i = 1; i <= 6; i++) {
            frequency[i] = 0;
        }

        // 骰子恢復為 1
        dicePanel.setNumber(1);

        // 更新資訊
        infoLabel.setText(
                "已擲 0 次，總和 0，平均 0.00"
```

```
        );

        // 更新點數統計
        statsLabel.setText(
                "1點：0次  2點：0次  3點：0次"
                + "  4點：0次  5點：0次  6點：0次"
        );
    }


    // ==========================================================
    // 自訂骰子面板
    // ==========================================================

    class DicePanel extends JPanel {

        private int number = 1;

        public DicePanel() {
            setBackground(Color.WHITE);
        }


        public void setNumber(int number) {

            this.number = number;

            repaint();
        }


        @Override
        protected void paintComponent(Graphics g) {

            super.paintComponent(g);

            Graphics2D g2 = (Graphics2D) g.create();

            // 開啟反鋸齒
            g2.setRenderingHint(
                    RenderingHints.KEY_ANTIALIASING,
```

```
                RenderingHints.VALUE_ANTIALIAS_ON
        );


        // ===== 骰子大小 =====
        int size = 130;

        int x = (getWidth() - size) / 2;
        int y = (getHeight() - size) / 2;


        // ===== 骰子陰影 =====
        g2.setColor(new Color(180, 180, 180));

        g2.fillRoundRect(
                x + 6,
                y + 8,
                size,
                size,
                30,
                30
        );


        // ===== 骰子本體 =====
        g2.setColor(Color.WHITE);

        g2.fillRoundRect(
                x,
                y,
                size,
                size,
                30,
                30
        );


        // ===== 骰子邊框 =====
        g2.setColor(new Color(150, 150, 150));

        g2.setStroke(new BasicStroke(2));
```

```
g2.drawRoundRect(
        x,
        y,
        size,
        size,
        30,
        30
);


// ===== 點的位置 =====
int pipSize = 20;

int left = x + 35;
int center = x + size / 2 - pipSize / 2;
int right = x + size - 55;

int top = y + 30;
int middle = y + size / 2 - pipSize / 2;
int bottom = y + size - 50;


// ===== 根據點數設定顏色 =====

if (number == 1) {

    g2.setColor(Color.RED);

} else if (number == 6) {

    g2.setColor(Color.GREEN);

} else {

    g2.setColor(Color.BLACK);
}


// ===== 根據點數畫點 =====
```

```
        switch (number) {

            case 1:
                drawPip(g2, center, middle, pipSize);
                break;

            case 2:
                drawPip(g2, left, top, pipSize);
                drawPip(g2, right, bottom, pipSize);
                break;

            case 3:
                drawPip(g2, left, top, pipSize);
                drawPip(g2, center, middle, pipSize);
                drawPip(g2, right, bottom, pipSize);
                break;

            case 4:
                drawPip(g2, left, top, pipSize);
                drawPip(g2, right, top, pipSize);
                drawPip(g2, left, bottom, pipSize);
                drawPip(g2, right, bottom, pipSize);
                break;

            case 5:
                drawPip(g2, left, top, pipSize);
                drawPip(g2, right, top, pipSize);
                drawPip(g2, center, middle, pipSize);
                drawPip(g2, left, bottom, pipSize);
                drawPip(g2, right, bottom, pipSize);
                break;

            case 6:
                drawPip(g2, left, top, pipSize);
                drawPip(g2, right, top, pipSize);

                drawPip(g2, left, middle, pipSize);
                drawPip(g2, right, middle, pipSize);

                drawPip(g2, left, bottom, pipSize);
                drawPip(g2, right, bottom, pipSize);
```

```
                    break;
            }

            g2.dispose();
        }


        // ===== 畫骰子上的一個點 =====
        private void drawPip(
                Graphics2D g2,
                int x,
                int y,
                int size
        ) {

            g2.fillOval(x, y, size, size);
        }
    }


    // ===== 主程式 =====
    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
            new HW2();
        });
    }
}
```

三、這次最重要的新增程式觀念

我挑出幾個你之後考試可能會用到的地方。

1. 如何讓骰子點數變色？

原本你的程式是：

```
g2.setColor(Color.BLACK);
```

所以不管骰子是多少，點點都會是黑色。

現在改成：

```
if (number == 1) {
    g2.setColor(Color.RED);
} else if (number == 6) {
    g2.setColor(Color.GREEN);
} else {
    g2.setColor(Color.BLACK);
}
```

這裡使用的是 `if else` 條件判斷。

2. 如何連續擲骰十次？

```
private void rollTenTimes() {

    for (int i = 0; i < 10; i++) {

        int dice = random.nextInt(6) + 1;

        updateData(dice);
    }
}
```

這裡最重要的是：

- `for`：重複執行程式。
- `i = 0`：從第 0 次開始。
- `i < 10`：執行到第 9 次。
- `i++`：每次增加 1。

因此總共執行十次。

3. 如何紀錄每個點數出現幾次？

```
private int[] frequency = new int[7];
```

這裡使用陣列紀錄出現次數。

為什麼是 7？

因為我們希望直接使用 `frequency[1]` 到 `frequency[6]`，分別代表骰子的六種點數。

例如：

```
frequency[dice]++;
```

假設這次骰出 6，就會執行：

```
frequency[6]++;
```

代表六點出現次數增加一次。

4. 為什麼要另外寫 updateData()？

這是這次程式修改很重要的地方。

原本只有一個 `rollDice()`，現在又增加了 `rollTenTimes()`。

如果兩個方法都各自寫一遍統計、更新畫面、計算平均，就會有很多重複程式碼。

所以我們把共同的工作放進：

```
private void updateData(int dice)
```

這樣不管是單次擲骰還是連骰十次，都可以呼叫同一個方法。

四、這次新增功能的程式架構



擲一次

連骰十次

重新開始

↓

updateData(dice)

更新次數、總和、平均、點數統計及骰子畫面

↓

DicePanel

paintComponent() 負責繪製骰子

我建議你下一步可以再增加一個「自訂連骰次數」功能，例如讓使用者輸入 5、10、20、100 次，而不是只能固定連骰十次。這樣整個作業會更有完整度。